

ÁP DỤNG DFS , BFS ĐẾM THÀNH PHẦN LIÊN THÔNG TRÊN ĐỒ THỊ

1. Một số bài toán áp dụng

Bài toán 1: Vùng dân cư liên thông (Trích đề thi chọn HSG lớp 12 Tỉnh Hà Tĩnh năm học 2012 - 2013)

N địa điểm dân cư có số hiệu là 1..N. Giữa 2 địa điểm dân cư bất kỳ trong N địa điểm dân cư đó có thể có hoặc không có đường đi trực tiếp tới nhau, nếu có thì chỉ có tối đa là một đường đi. Ta định nghĩa một vùng dân cư liên thông là tập hợp các địa điểm dân cư mà với 2 địa điểm dân cư bất kỳ của vùng đều đi được tới nhau trực tiếp hoặc thông qua các địa điểm dân cư khác của vùng.

Hãy xác định số lượng vùng dân cư liên thông của N địa điểm dân cư đã nêu.

Dữ liệu vào: là tệp văn bản DANCU.INP có cấu trúc:

- Dòng đầu tiên ghi hai số N, M ($1 < N \leq 5000$).
- M dòng tiếp theo, mỗi dòng ghi hai số cách nhau ít nhất một ký tự trống là số hiệu của hai địa điểm dân cư mà có đường đi trực tiếp tới nhau.

Dữ liệu ra: là tệp DANCU.OUT ghi một số duy nhất là số lượng vùng dân cư liên thông của N địa điểm dân cư đã được mô tả trong tệp dữ liệu vào.

Ví dụ:

Tệp DANCU.INP	Tệp DANCU.OUT
5 3 1 2 4 5 3 2	2

Ý tưởng thuật toán:

- Chúng ta xem mỗi điểm dân cư là một đỉnh của một đồ thị vô hướng. Lúc này, đường đi của 2 điểm dân cư i và j chính là đường đi của hai đỉnh i và j của đồ thị.
- Áp dụng thuật toán "DFS" hoặc "BFS" xác định số thành phần liên thông trên đồ thị chính là số thành phần liên thông của N điểm dân cư.

Chương trình tham khảo như sau:

Chương trình dùng thuật toán BFS	Chương trình dùng thuật toán DFS
<pre>#include <bits/stdc++.h> using namespace std; int n , m ; vector <int> ke[1003]; bool DD[1003] ; void nhap() {</pre>	<pre>#include <bits/stdc++.h> using namespace std; int n , m ; vector <int> ke[1003]; bool DD[1003] ; void nhap() {</pre>

<pre> freopen("DANCU.INP","r",stdin); freopen("DANCU.OUT","w",stdin); cin >> n >> m ; for (int i = 0 ; i< m ; i++) { int x , y ; cin >> x >> y ; ke[x].push_back(y) ; ke[y].push_back(x) ; } memset(DD , false , sizeof(DD)) ; } //===== void BFS(int u) { queue <int> q ; q. push(u) ; DD[u]=true; while (!q.empty()) { int v = q. front() ; q.pop() ; for (int x: ke[v]) { if (!DD[x]) { q.push(x) ; DD[x]=true; } } } } //===== void tplt() { int dem = 0; for(int i = 1; i<=n ; i++) { if (!DD[i]) { dem++ ; BFS(i) ; } } cout<< dem; } //===== int main() { nhap(); tplt(); return 0; } </pre>	<pre> freopen("DANCU.INP","r",stdin); freopen("DANCU.OUT","w",stdin); cin >> n >> m ; for (int i = 0 ; i< m ; i++) { int x , y ; cin >> x >> y ; ke[x].push_back(y) ; ke[y].push_back(x) ; } memset(DD , false , sizeof(DD)) ; } //===== void DFS(int u) { DD[u]=true; for (int x: ke[u]) { if (!DD[x]) { DFS(x) ; } } } //===== void tplt() { int dem = 0; for(int i = 1; i<=n ; i++) { if (!DD[i]) { dem++ ; DFS(i) ; } } cout<< dem; } //===== int main() { nhap(); tplt(); return 0; } </pre>
---	--

Bài toán 2: Bệnh SARS (Trích đề thi Olympic Tin học sinh viên lần thứ XII, 2003)

Một cơ quan có N nhân viên được đánh số thứ tự từ 1 đến N. Mỗi người có một phòng làm việc riêng của mình. Do nhu cầu công việc, hàng ngày mỗi nhân viên có thể phải tiếp xúc với một số nhân viên khác. Vào một ngày làm việc bình thường, có một nhân viên bị nhiễm bệnh SARS, nhưng do không biết nên người này vẫn đi làm. Đến cuối ngày làm việc người ta mới phát hiện ra người nhiễm bệnh SARS đầu tiên, Khả năng lây lan của SARS rất nhanh chóng: một người nhiễm bệnh nếu tiếp xúc với một người khác có thể sẽ truyền bệnh cho người này.

Yêu cầu

Hãy giúp các bác sỹ kiểm tra xem cuối ngày hôm đó, có tối đa bao nhiêu người có thể sẽ nhiễm bệnh và đó là những người nào để còn cách ly. Người có tiếp xúc với người nhiễm bệnh được coi là người nhiễm bệnh.

Dữ liệu: vào từ file văn bản SARS.INP.

- Dòng đầu tiên ghi 2 số tự nhiên N và K ($1 < N \leq 250$, $1 \leq K \leq N$) tương ứng là số lượng người làm việc trong toà nhà và số hiệu của nhân viên đã nhiễm SARS đầu tiên.

- Dòng thứ I trong N dòng tiếp theo ghi danh sách những người có tiếp xúc với người thứ I theo cách sau: số đầu tiên J của dòng là tổng số nhân viên đã gặp người thứ I , tiếp theo là J số tự nhiên lần lượt là số hiệu của các nhân viên đó. Nếu $J = 0$ có nghĩa rằng không ai đã tiếp xúc với người thứ I .

Kết quả : ghi ra file văn bản SARS.OUT như sau:

- Dòng đầu tiên ghi số S là tổng số người có thể bị lây nhiễm SARS.
- Dòng thứ 2 liệt kê tất cả các người có thể bị lây nhiễm SARS cần cách ly, danh sách cần được đưa ra theo thứ tự của số hiệu nhân viên nhiễm bệnh.

Trong file dữ liệu vào và file kết quả, các số trên một dòng cách nhau ít nhất một dấu cách.

Ví dụ:

SARS.INP	SARS.OUT
5 1	3
2 2 3	1 2 3
2 1 3	
1 2	
1 5	
1 4	

Ý tưởng giải thuật:

- Chúng ta xem mỗi nhân viên của cơ quan là một đỉnh của một đồ thị vô hướng. Lúc này ta quy ước hai đỉnh i và j được gọi là có đường nối nếu như hai nhân viên i và j có tiếp xúc với nhau.

- Áp dụng thuật toán "DFS" hoặc "BFS" xuất phát từ đỉnh k (người đầu tiên bị nhiễm bệnh). Tiến hành duyệt và đánh dấu đã duyệt bắt đầu từ đỉnh k , tìm đến đâu đếm và đánh dấu số nhân viên bị lây nhiễm đến đó đồng thời dùng **vector res** lưu lại mã số của nhân viên bị nhiễm. Sau khi duyệt hết các đỉnh liên thông với k (người có tiếp xúc với người

nhằm bệnh bắt đầu từ người thứ k) chúng ta sẽ tìm ra được tổng số người tối đa nhiễm bệnh và tên từng người đó.

Chương trình tham khảo dùng DFS, có thể thay bằng BFS

Chương trình dùng thuật toán BFS	Chương trình dùng thuật toán DFS
<pre> #include <bits/stdc++.h> using namespace std; const int N=255; int n , k , dem=0; vector <int> ke[N], res; bool DD[N] ; void nhap() { freopen("SARS.INP","r",stdin); //freopen("SARS.OUT","w",stdin); cin >> n >> k ; int j,x ; for (int i = 1 ; i<= n ; i++) { cin >> j ; for(int l=1;l<=j;l++){ cin>>x; ke[i].push_back(x) ; } } memset(DD , false , sizeof(DD)) ; } //===== void BFS(int u) { queue <int> q ; q. push(u) ; DD[u]=true; while (!q.empty()) { int v = q. front() ; q.pop() ; dem++; res.push_back(v); for (int x: ke[v]) { if (!DD[x]) { q.push(x); DD[x]=true; } } } } //===== int main() { nhap(); BFS(k); cout<< dem<<endl; sort(res.begin(),res.end()); for(int i : res) cout<<i<<" "; return 0; } </pre>	<pre> #include <bits/stdc++.h> using namespace std; const int N=255; int n , k , dem=0; vector <int> ke[N], res; bool DD[N] ; void nhap() { freopen("SARS.INP","r",stdin); //freopen("SARS.OUT","w",stdin); cin >> n >> k ; int j,x ; for (int i = 1 ; i<= n ; i++) { cin >> j ; for(int l=1;l<=j;l++){ cin>>x; ke[i].push_back(x) ; } } memset(DD , false , sizeof(DD)) ; } //===== void DFS(int u) { DD[u]=true; dem++; res.push_back(u); for (int v: ke[u]) { if (!DD[v]) { DFS(v); } } } //===== int main() { nhap(); DFS(k); cout<< dem<<endl; sort(res.begin(),res.end()); for(int i : res) cout<<i<<" "; return 0; } </pre>

2. Bài tập tự luyện

Bài 1: Đếm nhóm bạn trong Hội trại (Trích đề thi chọn HSG lớp 12 Tỉnh Quảng Bình năm học 2012 - 2013)

Trong một Hội trại hè do Tỉnh Đoàn tổ chức, có N học sinh tham gia, trong đó, có một số học sinh quen nhau. Một số học sinh được gọi là cùng một nhóm bạn, nếu bất kì một học sinh nào thuộc nhóm đều có quen ít nhất một học sinh khác trong cùng nhóm đó.

Yêu cầu: Hãy đếm xem có bao nhiêu nhóm bạn trong N học sinh tham gia Hội trại.

Dữ liệu vào: Cho trong file văn bản NHOMBAN.INP, có cấu trúc như sau:

- Dòng 1: Ghi số nguyên dương N , là số lượng học sinh tham gia Hội trại.

($1 \leq N \leq 1000$).

- Trong N dòng tiếp theo: Mỗi dòng ghi N số nguyên dương $a[i,j]$ với ý nghĩa:

$a[i,j] = 1$ nếu học sinh i quen học sinh j (với $i \neq j$).

$a[i,j] = 0$ nếu học sinh i không quen học sinh j (với $i \neq j$).

$a[i,i] = 1$ (học sinh i được xem là quen bản thân nó).

Các số trên cùng một dòng được ghi cách nhau ít nhất một dấu cách.

Dữ liệu ra: Ghi ra file văn bản NHOMBAN.OUT, theo cấu trúc như sau:

- Dòng 1: Ghi số nguyên dương K , là số lượng nhóm bạn tìm được trong N học sinh tham gia Hội trại.

Ví dụ:

NHOMBAN.INP	NHOMBAN.OUT
5	2
1 0 0 1 1	
0 1 1 0 0	
0 1 1 0 0	
1 0 0 1 1	
1 0 0 1 1	

Ý tưởng giải thuật: Có thể giải bài toán theo nhiều cách. Cách giải sau đây áp dụng thuật toán tìm kiếm theo chiều rộng hoặc thuật toán tìm kiếm theo chiều sâu.

Với mỗi học sinh tham gia hội trại được biểu diễn tương ứng là một đỉnh của một đồ thị vô hướng. Yêu cầu của bài toán được chuyển thành tìm số thành phần liên thông của đồ thị.

Giả sử ta đang tìm các bạn học sinh thuộc một nhóm bạn nào đó, ví dụ nhóm 1, lúc này ta tiến hành tìm kiếm từ một học sinh x bất kỳ trong số N học sinh tham gia hội trại và đánh dấu học sinh này là đã tham gia vào một nhóm 1. Từ học sinh x ta tìm và đánh dấu những học sinh nào có quen với học sinh x và có quen nhau đưa vào nhóm 1.

Nếu tất cả học sinh đều được đánh dấu thì kết luận có một nhóm bạn, ngược lại ta tiếp tục tìm kiếm nhóm bạn khác bắt đầu từ một bạn học sinh chưa đánh dấu. Quá trình tìm nhóm bạn kết thúc khi tất cả các học sinh đều được đánh dấu.

Số lần gọi thuật toán BFS cho biết số nhóm bạn cần tìm.

Bài 2: Đếm quần đảo (Trích đề thi HSG lớp 12 tỉnh Kiên Giang năm học 2011-2012)

Biển đông của nước ta có rất nhiều đảo chìm, đảo nổi. Để thuận tiện cho việc canh giữ và quản lý các đảo, bộ đội Hải Quân đã đánh số hiệu các đảo từ 1..n, đồng thời xây dựng đường đi qua lại giữa các đảo sao cho luôn có ít nhất một đường đi từ đảo này qua đảo kia tạo thành một quần đảo (quần đảo là một hoặc nhiều đảo có đường đi nối nhau). Khi nước thủy triều lên, xuống thì số lượng các đảo hoặc đường đi có thể giảm hoặc tăng. Tại đài quan sát luôn có những số liệu tương ứng với thời điểm hiện tại của mực nước biển.

Yêu cầu: với một bộ số liệu tại một thời điểm, hãy giúp các chiến sĩ xác định có bao nhiêu quần đảo tồn tại.

Dữ liệu: vào từ file văn bản QUANDAO.INP có cấu trúc như sau:

- Dòng đầu gồm 2 số nguyên n, m ($n, m \leq 1000$)
- m dòng tiếp theo, mỗi dòng 2 số nguyên chỉ 2 đảo có đường đi nối nhau

Các số trên cùng 1 dòng ghi cách nhau ít nhất 1 ký tự trắng.

Kết quả: Ghi ra file văn bản QUANDAO.OUT

Mỗi dòng là số hiệu các đảo của một quần đảo tìm được

Ví dụ :

QUANDAO.INP	QUANDAO.OUT
12 9	Quan dao thu 1 : 1, 2, 3, 4, 5
1 3	Quan dao thu 2 : 6, 7, 8
1 4	Quan dao thu 3 : 9, 10, 11, 12.
1 5	
2 4	
6 7	
6 8	

9 10	
9 11	
11 12	

Ý tưởng

giải thuật :

Ta xem mỗi đảo của quần đảo là một đỉnh của một đồ thị vô hướng. Khi đó đường đi giữa đảo này đến đảo kia được kí hiệu bằng cạnh nối giữa đỉnh này và đỉnh kia của một đồ thị. Bài toán được đưa về tìm số thành phần liên thông của đồ thị (tương ứng với tìm số quần đảo đã tồn tại), mỗi thành phần liên thông gồm những đỉnh nào (tương ứng với mỗi quần đảo gồm những đảo nào). Vận dụng thuật toán "tìm kiếm theo chiều rộng" trên đồ thị vô hướng chúng ta sẽ giải quyết bài toán một cách dễ dàng.

Bài 3: Cụm kết nối (Trích đề thi chọn HSG lớp 12 Tỉnh Hà Tĩnh năm học 2022 - 2023)

Trong đồ thị vô hướng, nếu tập hợp một hoặc nhiều đỉnh mà giữa hai đỉnh bất kỳ đều có đường đi đến nhau là một cụm kết nối, thì khi ta xóa đi một đỉnh nào đó của đồ thị vô hướng ta có thể nhận được một hoặc nhiều cụm kết nối (cụm kết nối có thể chỉ có 1 đỉnh).

Cho một đồ thị vô hướng gồm n đỉnh đánh số từ 1 đến n và m cạnh (mỗi cạnh là một đường nối giữa 2 đỉnh). Nếu ta xóa đi một đỉnh nào đó của đồ thị, số cụm kết nối của đồ thị có thể thay đổi. Nhiệm vụ của bạn là với mỗi đỉnh, hãy tính xem nếu ta xóa đỉnh đó đi thì đồ thị mới nhận được có bao nhiêu cụm kết nối.

Dữ liệu: Vào từ tệp văn bản CKN.INP

- Dòng đầu chứa 2 số nguyên dương n, m ($n \leq 20000$; $m \leq 50000$)
- m dòng sau, mỗi dòng chứa 2 số nguyên dương u, v thể hiện có cạnh nối giữa hai đỉnh u và v .

Kết quả: Ghi ra tệp văn bản CKN.OUT gồm n dòng, dòng thứ j ($1 \leq j \leq n$) cho biết số cụm kết nối của đồ thị nếu ta xóa đi đỉnh j .

Ràng buộc:

- 80% số test ứng với 80% số điểm có $n \leq 1000$; $m \leq 2000$;
- 20% số test ứng với 20% số điểm có $1000 < n \leq 20000$; $2000 < m \leq 50000$;

Ví dụ:

CKN.INP	CKN.OUT
5 4	1

1 2	3
2 4	1
2 5	2
3 4	1

Bài 4:

<https://oj.vnoi.info/problem/message>